



Trabajo Práctico Grupal: Money Laundering Analysis

[75.74/TA050] Sistemas distribuidos I

Autores:

Nombre	DNI	Padrón	Correo electrónico
Bustamante Gonzalo Manuel	43.086.565	105.011	gbustamante@fi.uba.ar
Perez Esnaola Lucas	43.630.933	107.990	lpereze@fi.uba.ar
Nemirovsky Federico	43.876.361	108.560	fnemirovsky@fi.uba.ar

Índice

Índice.....	2
Introducción.....	3
Requerimientos funcionales.....	3
Diagrama de tareas.....	4
Diagrama de tareas de Query 1.....	5
Diagrama de tareas de Query 2.....	6
Diagrama de tareas de Query 3.....	7
Diagrama de tareas de Query 4.....	8
Diagrama de tareas de Query 5.....	9
Vista de procesos.....	10
Diagrama de secuencia de cliente con sistema.....	10
Diagrama de actividad de Q4.....	12
Vista de desarrollo.....	14
Diagrama de paquetes.....	14
Vista de física.....	15
Diagrama de despliegue.....	15
Diagrama de Robustez.....	16
Arquitectura de monitoreo y elección de líder.....	18
<u>Tolerancia a Fallas.....</u>	<u>18</u>
Listado de tareas.....	19

Introducción

Este documento apunta a ilustrar y explicar la arquitectura del sistema realizado para la asignatura Sistemas Distribuidos I de la Facultad de Ingeniería de la Universidad de Buenos Aires. En el mismo se encontrarán las decisiones tomadas, así como los casos de uso y los diagramas solicitados.

Requerimientos funcionales

Como requerimientos funcionales del sistema se solicitó la realización de una serie de objetivos que se detallan a continuación a partir de un conjunto de datos para el análisis de lavado de dinero. Los requerimientos son:

- **Solicitud 1 (Q1) - Transacciones de poca cantidad de dinero:** Se pide encontrar las cuentas de origen, cuentas de destino y monto para transacciones en dólares (USD) menores a 50.
- **Solicitud 2 (Q2) - Máximos enviados en cada banco:** Se piden nombres de bancos, cuentas de origen y montos de las máximas transacciones en dólares (USD) de cada banco.
- **Solicitud 3 (Q3) - Transferencias de montos más pequeños que el promedio:** Se piden cuentas de origen y montos de transacciones en dólares (USD) en el período 06/09/2022 - 15/09/2022 con monto menor a 1 centésimo del promedio encontrado para el mismo formato de pago en el período 01/09/2022 - 05/09/2022.
- **Solicitud 4 (Q4) - Scatter-gather con un intermediario:** Cuentas que cumplan con el patrón *scatter-gather* con una sola cuenta de separación. Se toman cuentas que hayan realizado transferencias en dólares (USD) hacia al menos 5 cuentas distintas dentro del período 01/09/2022 - 05/09/2022.
- **Solicitud 5 (Q5) - Transacciones convertidas:** Cantidad de transacciones del período 01/09/2022 - 05/09/2022 con formato de pago *Wire* o *ACH* cuyo monto convertido a dólares (USD) sea menor a 1. Para este punto se utilizará una API provista por la cátedra.

Diagrama de tareas

A continuación se muestra un diagrama en forma de grafo dirigido acíclico (DAG) que muestra los flujos de los datos a través del sistema y cómo se llega a los resultados finales requeridos.

Los colores permiten distinguir un poco mejor las operaciones que se realizan en cada paso:

- Azul: Se realizan operaciones de I/O con elementos por fuera del sistema.
- Verde: Se realiza un filtro sobre la información.
- Naranja: Se realiza una agregación sobre datos parciales.
- Rojo: Se realiza una fusión de datos parciales para obtener un resultado global.
- Amarillo: Se realiza una operación que prepara la información específicamente para otra etapa.

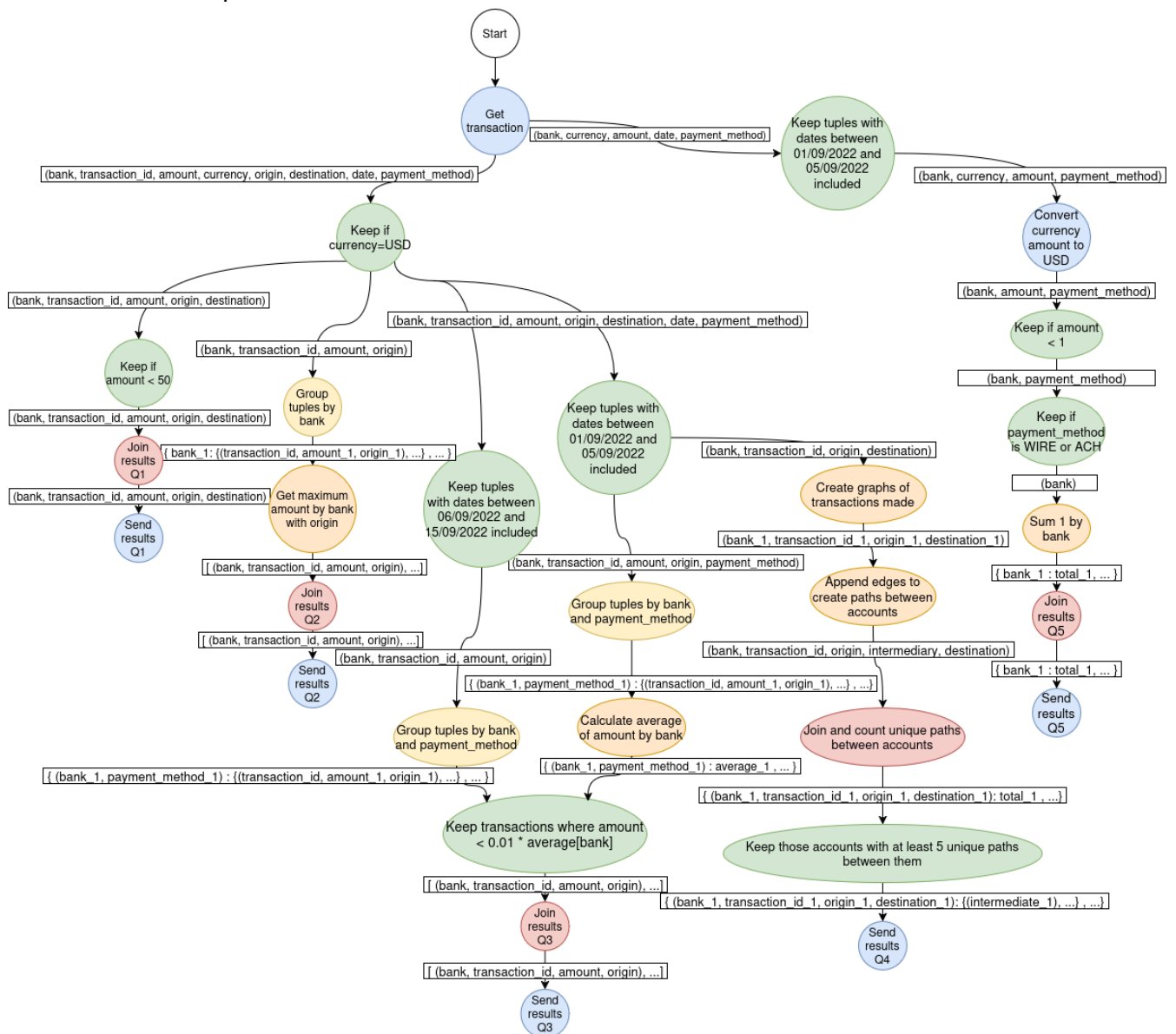


Diagrama de tareas de Query 1

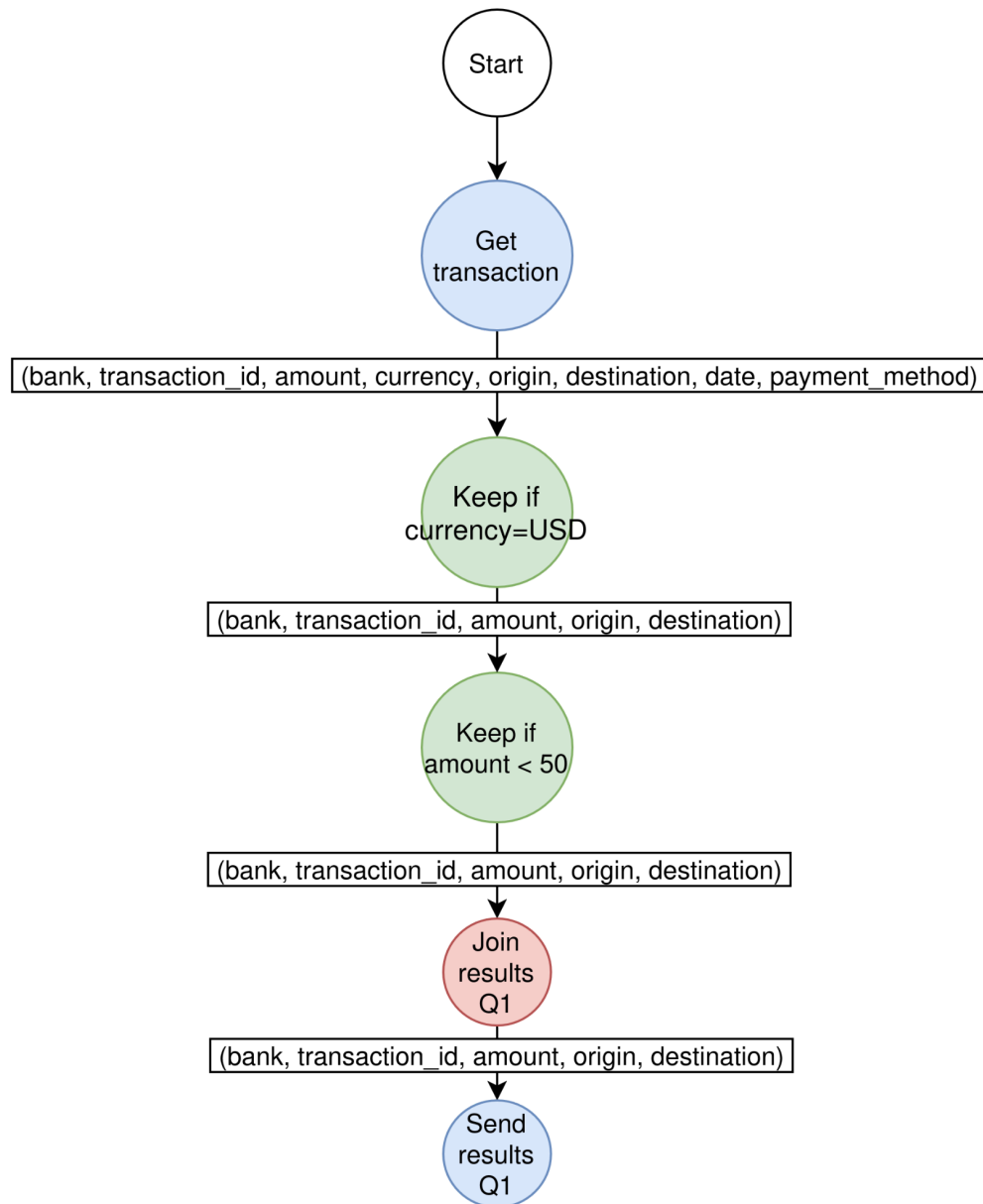


Diagrama de tareas de Query 2

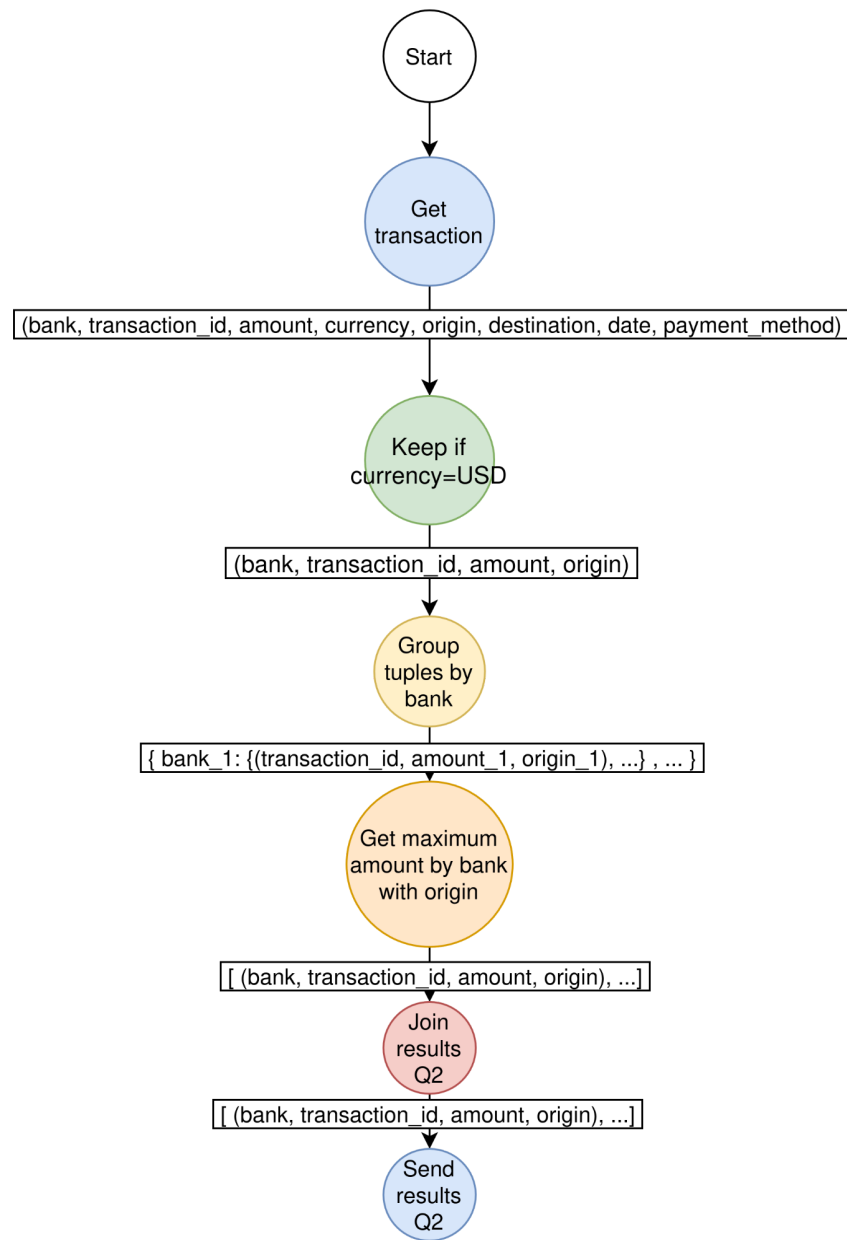


Diagrama de tareas de Query 3

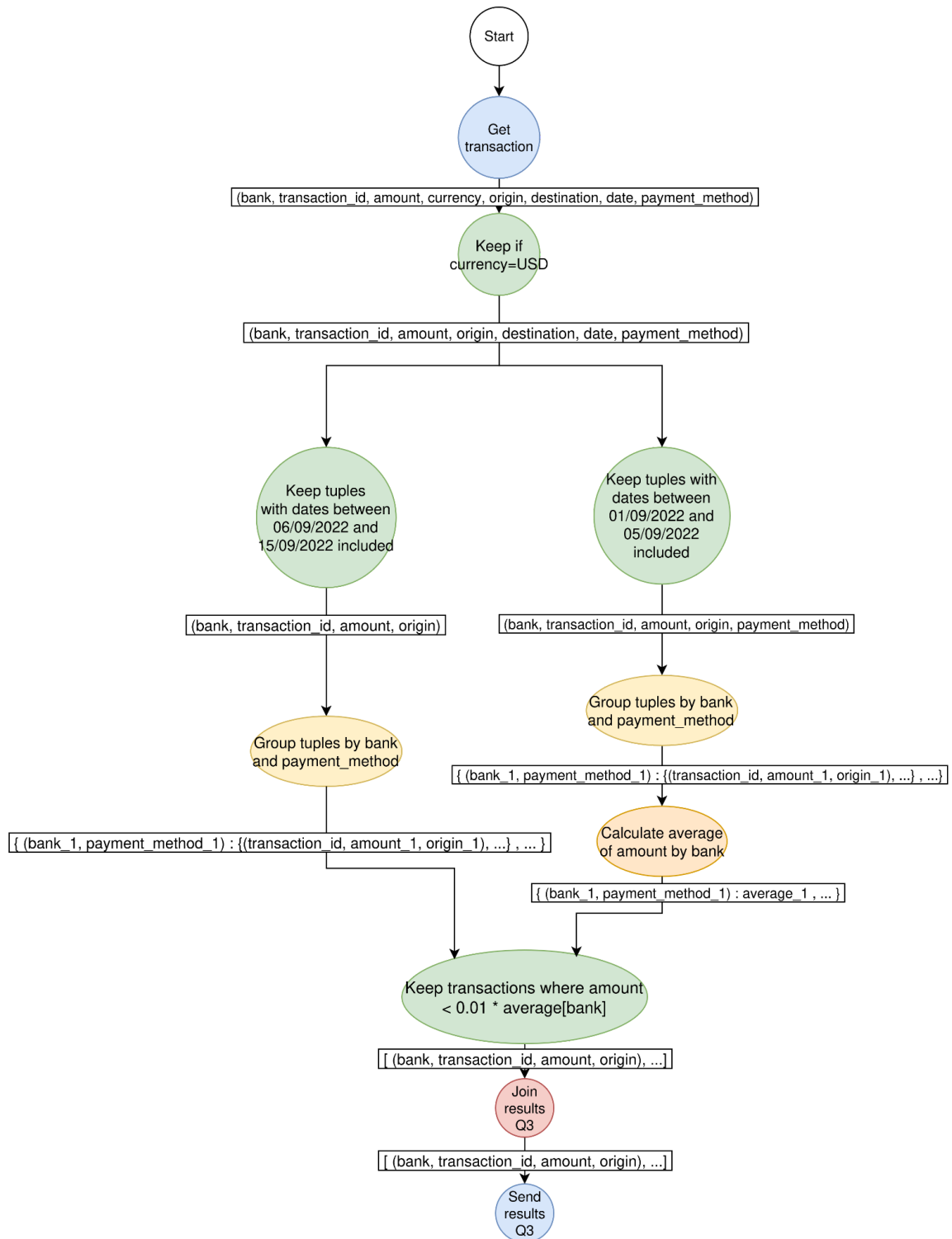


Diagrama de tareas de Query 4

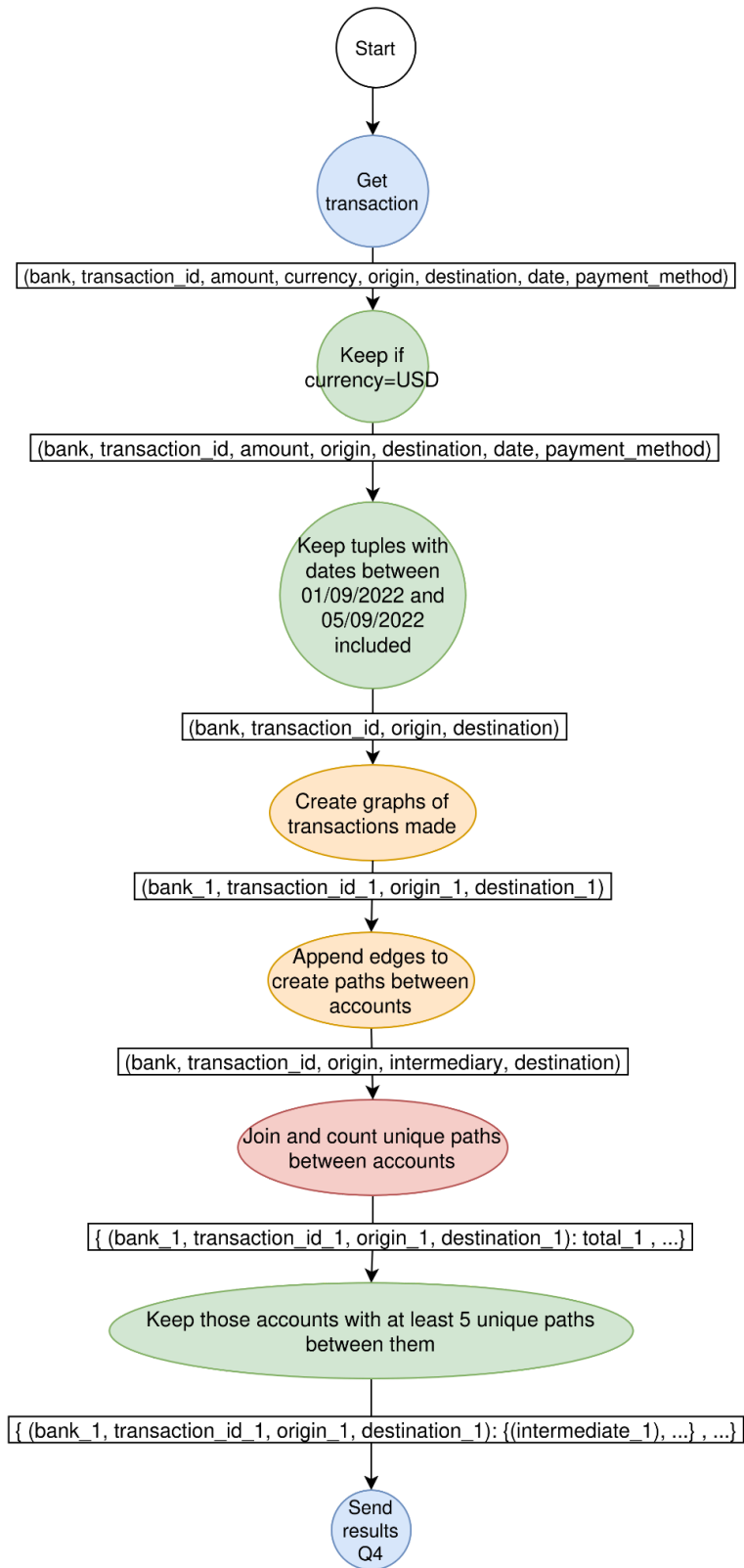
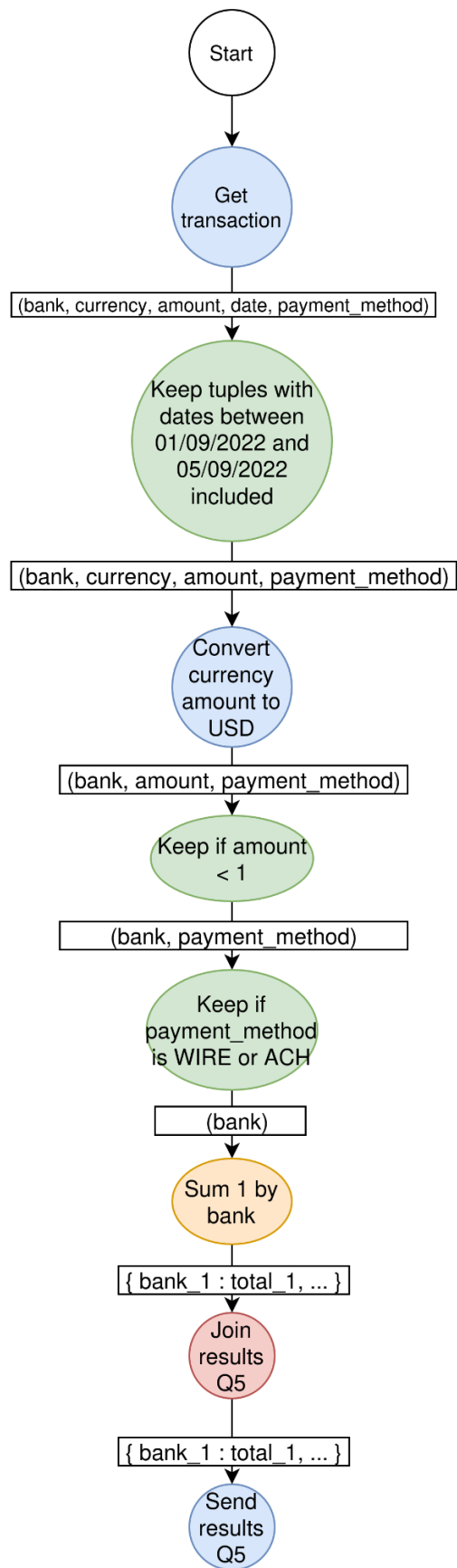


Diagrama de tareas de Query 5



Vista de procesos

Diagrama de secuencia de cliente con sistema

El cliente necesariamente debe comunicarse con el sistema de alguna manera para enviar los datos que posee en dos archivos: uno de cuentas bancarias y otro de transacciones realizadas. Para ello se define un protocolo con el que el cliente se comunica con un gateway del sistema, el cual distribuye los datos que envía el cliente al interior del sistema.

El cliente al comunicarse con el *gateway* utiliza una conexión TCP utilizando el puerto 7659. Se busca conectarse enviando un mensaje y si es respondido con una afirmación se iniciará el envío de datos desde el cliente al sistema.

Primero se enviarán los datos de las cuentas bancarias en formas batches que son recibidos y de cada cuenta se extrae el ID y el nombre del banco al cual pertenece la cuenta, para crear una relación entre ellos. Esto se hace porque la *Query 2* pide los nombres de los bancos, entonces para poder luego reemplazar los identificadores por los nombres correspondientes a las transacciones de dicha *query* es que se hace este mapeo. Una vez que se envían todos los datos se le comunica al *gateway*.

Por último, y luego de enviar las cuentas bancarias, se envían las transacciones del archivo utilizando batches al *gateway*, quien luego envía al interior del sistema (es decir, a los distintos *workers*) los datos para su procesamiento. Es en este momento cuando se procesan los datos para generar los resultados de las *queries* solicitadas, de las cuales algunas necesitan esperar hasta el final del envío de los datos para tener un resultado (como la *query 3*) y otras permiten el envío de datos en formato de *streaming* (como la *query 1*). Una vez que todos los batches de transacciones son enviados entonces el cliente espera los resultados del procesamiento de los datos del sistema y que se confirme el procesamiento de todos los datos.

A continuación se muestra el diagrama de secuencia de la comunicación entre el cliente y el sistema:

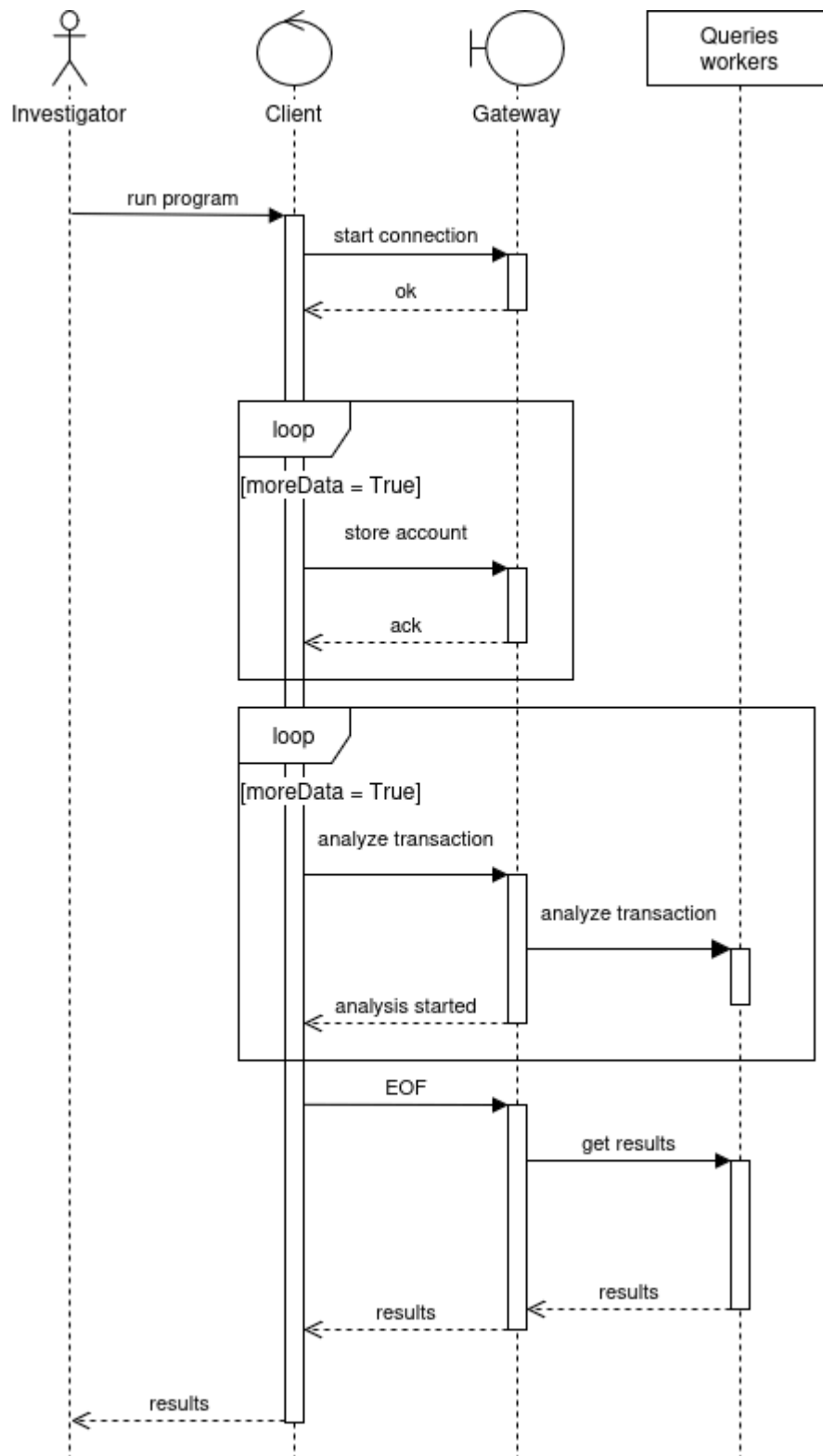
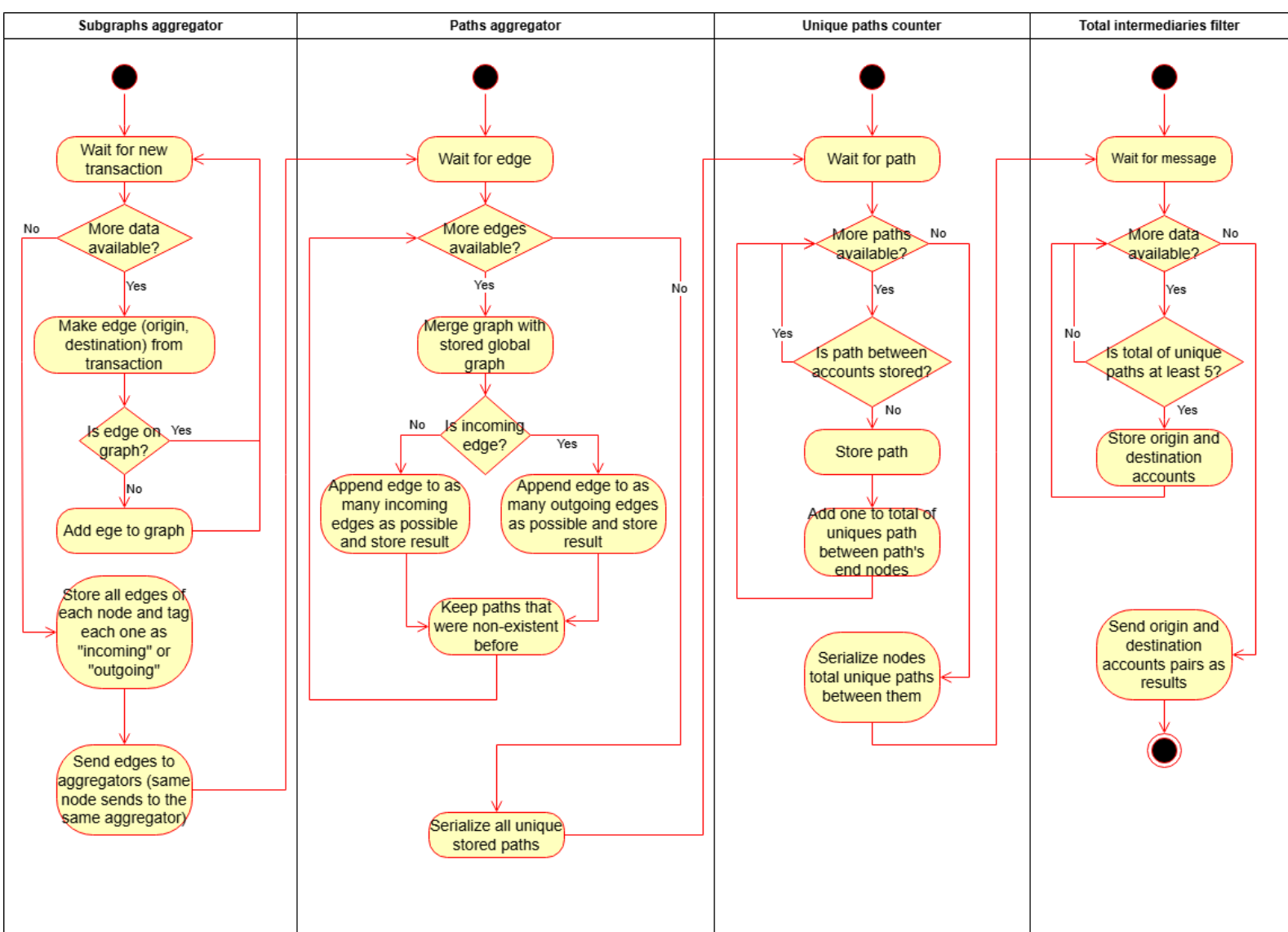


Diagrama de actividad de Q4

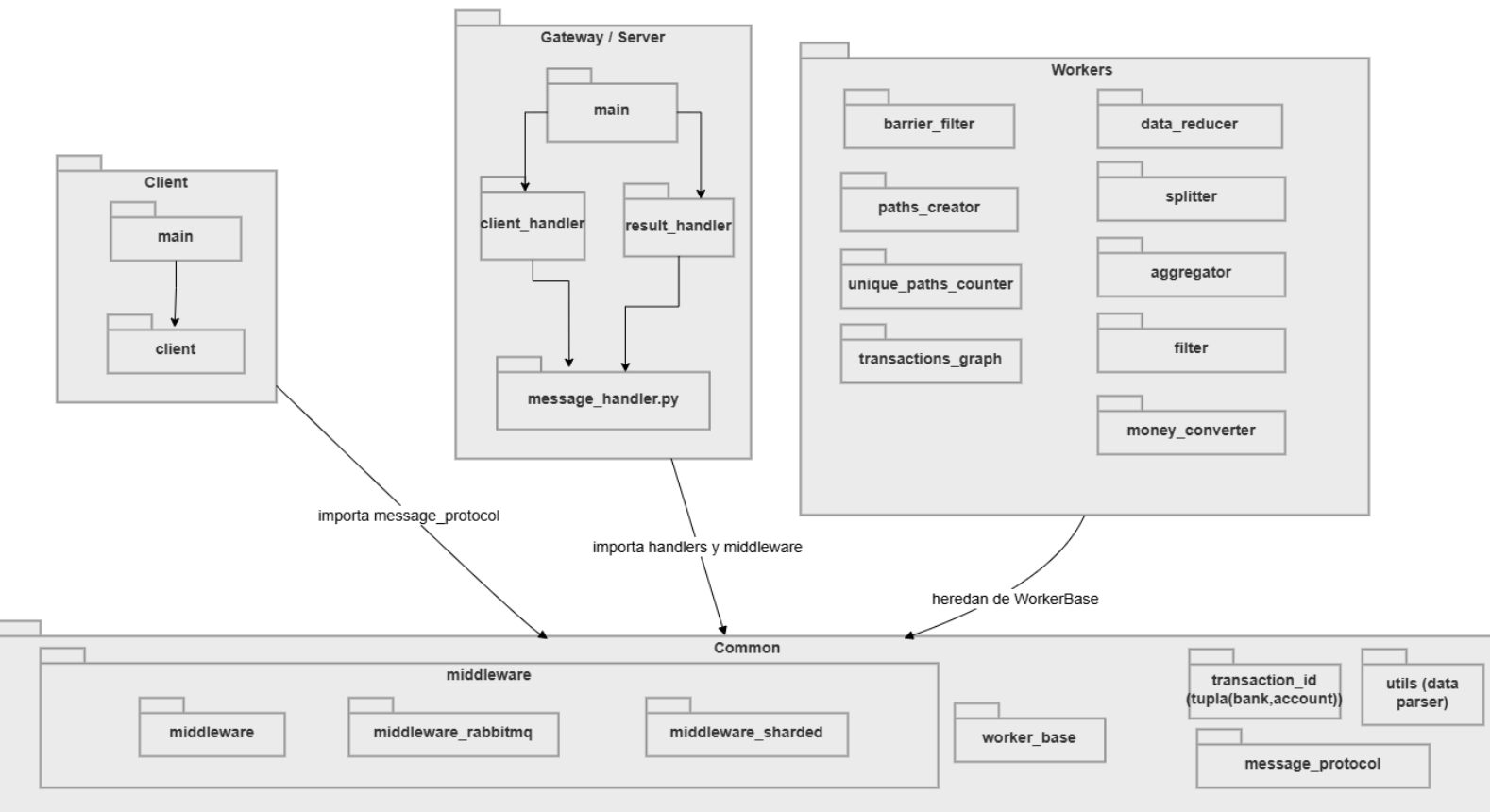
A continuación se muestra el armado de los grafos de transacciones realizadas para la *query* 4 (Q4). Se puso el foco en esta parte de la *query* ya que las demás son sencillas de entender.

El análisis de las cuentas que tengan el patrón *scatter-gather* inicia armando subgrafos en los agregadores de más a la izquierda del gráfico. Las transacciones se envían según el par de cuentas origen y destino a los agregadores de forma tal que los grafos compartan la menor cantidad de aristas posibles, de forma que no se repitan mensajes en la etapa siguiente. Cuando ya no hay más transacciones envían las aristas a los agregadores de los nodos origen y destino de forma tal que el agregador del nodo origen reciba la arista como de “saliente” del nodo y el agregador del nodo destino reciba la arista como “entrante” al nodo. Cabe aclarar que las aristas de un mismo nodo, sean entrantes o salientes, se enviarán al mismo agregador para poder tratar a cada nodo como un posible nodo intermediario del patrón *scatter-gather*.

En el agregador de “camino” lo que se hace es unir una arista entrante al nodo con otra saliente al nodo para poder generar un camino entre dos nodos con el nodo del cual entra y sale las aristas como intermediario. Esos caminos se van guardando y una vez que se recibieron todas las aristas, se envían dichos caminos al “contador de caminos únicos” que como su nombre indica cuenta la cantidad de caminos únicos entre dos nodos origen y destino. Por ejemplo, si existen los caminos $A \rightarrow C \rightarrow B$ y $A \rightarrow D \rightarrow B$ entonces hay dos caminos únicos entre A y B. Finalmente estos totales, junto con los nodos de “salida” y “llegada” se envían al “filtro de intermediarios” que cuenta la cantidad de caminos únicos entre dos nodos con un nodo entre ellos. Esto se hace porque si hay cinco o más caminos entonces eso quiere decir que hay cinco o más cuentas a las que se les envía dinero y que luego convergen en otra cuenta, como indica el patrón *scatter-gather*.



Vista de desarrollo



Este diagrama muestra cómo organizamos el código del sistema en partes más chicas para que cada una tenga una responsabilidad clara. En Client quedó todo lo relacionado con arrancar la aplicación y manejar la conexión TCP con el Gateway. Está compuesto por el `main`, que orquesta la ejecución, y `client`, que administra el envío de batches y la recepción de resultados. Mientras que `message_protocol` define el formato de los mensajes y centraliza la serialización y deserialización.

En Gateway/Server, el `main` levanta el servidor, `client_handler` recibe los batches del cliente y los publica en RabbitMQ, y `result_handler` consume los resultados desde RabbitMQ y arma la respuesta final para el cliente. El módulo `message_handler.py` centraliza la serialización y deserialización de los mensajes internos.

En Workers están los componentes que hacen el procesamiento distribuido, como `filter`, `money_converter`, `data_reducer`, `splitter`, `aggregator`, `barrier_filter`, `paths_creator`, `unique_paths_counter` y `transactions_graph`. Common reúne lo compartido por más de una parte del sistema, como `middleware`, `middleware_rabbitmq`, `middleware_sharded`, `worker_base`, `message_protocol`, `transaction_id` y `utils`. En particular, `WorkerBase` concentra la lógica común de consumo, publicación y manejo de EOF, mientras que `middleware_sharded` implementa el envío y la recepción sobre exchanges particionados por shard para repartir la carga entre múltiples instancias sin repetir el trabajo.

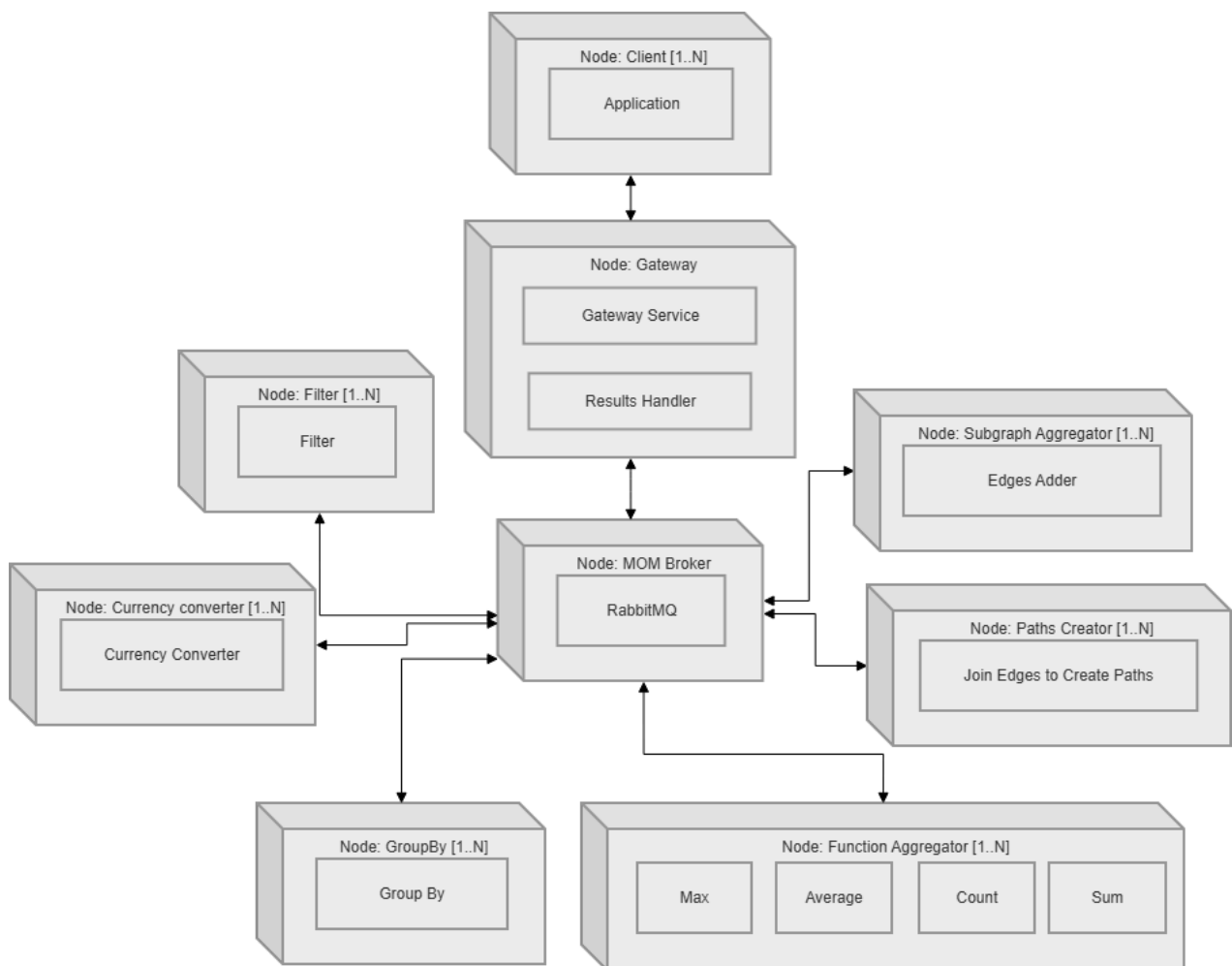
`WorkerBase` centraliza toda la parte de comunicación con RabbitMQ y el manejo del ciclo de vida del worker. Se encarga de crear el consumidor y el productor según la

configuración, leer variables de entorno, acumular resultados en buffers y resolver el envío de EOF. De esta manera, las subclases solo tienen que implementar el procesamiento concreto de cada mensaje, sin preocuparse por los detalles de conexión, publicación o consumo. Esto deja separada la lógica de infraestructura de la lógica de transformación de datos.

Vista de física

Diagrama de despliegue

El diagrama de despliegue muestra la distribución física de los componentes del sistema y cómo se comunican entre sí en tiempo de ejecución. La arquitectura se organiza alrededor de un Gateway central, que actúa como punto de entrada de los clientes y como coordinador de la comunicación con el resto de los nodos. Esto incluye el Gateway Service, que recibe los datos del cliente, los interpreta y los publica en RabbitMQ y el Results Handler, que consume los resultados finales desde RabbitMQ y los devuelve al cliente. La interacción entre componentes se realiza a través de un broker RabbitMQ, que desacopla emisores y receptores y permite el procesamiento asíncrono de los batches de datos.



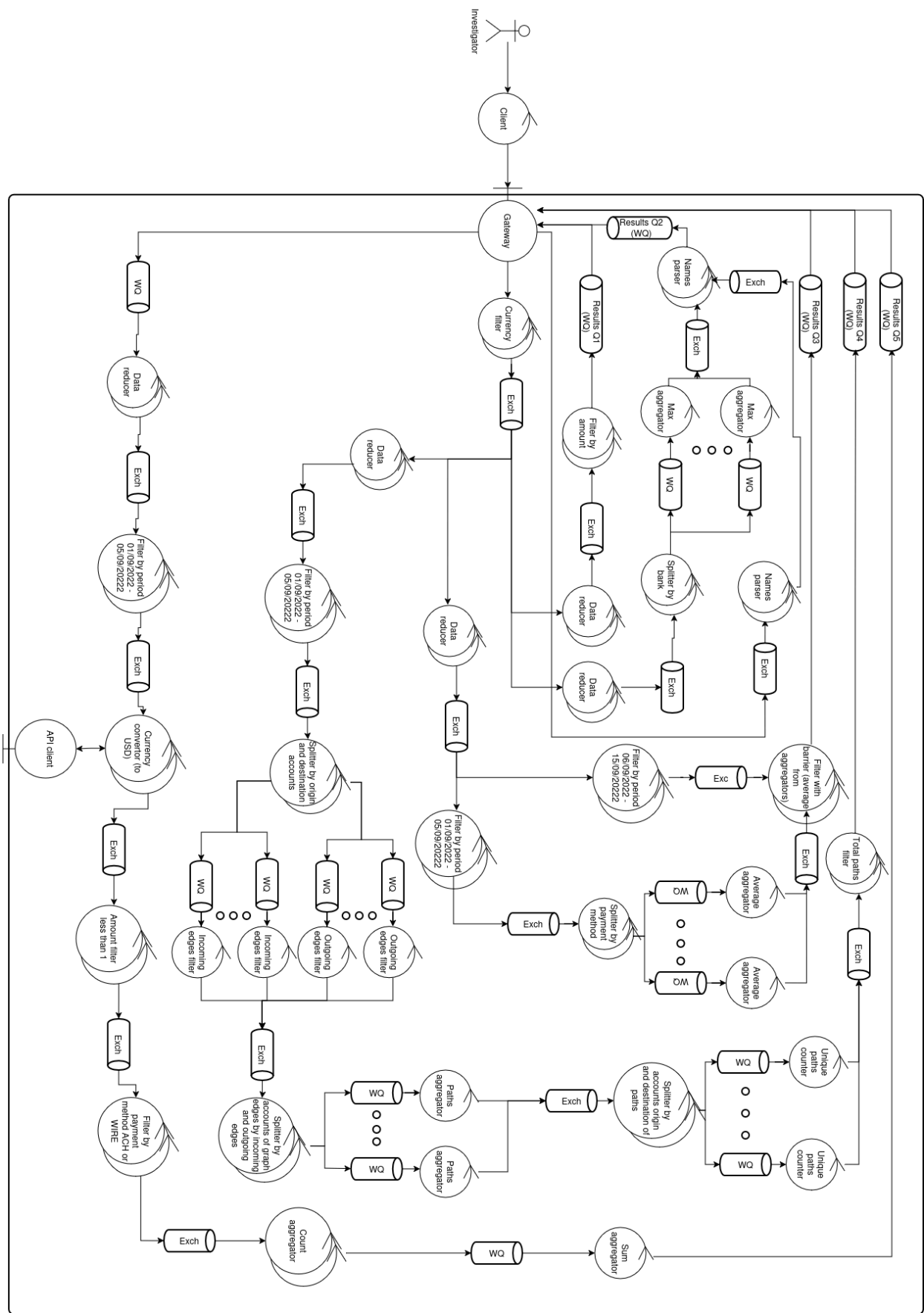
En la capa de procesamiento, los nodos se dividen por responsabilidad: Filter, Currency Converter, GroupBy, Joiner, Paths Creator, Subgraph Aggregator y Function Aggregator. Cada tipo de nodo puede ejecutarse en múltiples instancias, indicado por la notación [1..N], con el objetivo de permitir paralelismo y escalabilidad horizontal. A continuación se explican más en profundidad los nodos:

- **Client:** Ejecuta la aplicación cliente que lee los archivos de transacciones y los envía al Gateway mediante una conexión TCP directa, siendo el único nodo que no pasa por RabbitMQ para esta comunicación inicial.
- **Filter:** Aplica los distintos filtros del pipeline (por fecha, monto y formato de pago).
- **Converter:** Realiza la conversión de moneda a USD usando la cotización diaria.
- **GroupBy:** Agrupa transacciones por banco y formato de pago según la query.
- **Function aggregator:** Contiene cuatro procesos especializados con determinadas funciones para calcular: Max para Q2, Average para Q3, Count (de caminos únicos) para Q4 y Sum para Q5.
- **Subgraph aggregator:** Junta las transacciones recibidas y arma grafos teniendo como nodos las cuentas que participan de la transacción y las aristas la dirección del dinero en la transacción.
- **Paths Creator:** Toma las aristas de los nodos de los grafos de los **Subgraph aggregator** y unen nodos de aristas salientes a aristas entrantes para formar caminos entre dos nodos.

Diagrama de Robustez

Los siguientes diagramas permitirán visualizar la arquitectura de la solución propuesta para resolver los 5 requerimientos funcionales solicitados, asegurando que soporte cargas masivas de datos sin cuellos de botella ni puntos únicos de falla. A nivel general, la robustez de estos diagramas se apoya en tres decisiones clave de diseño:

- Desacoplamiento mediante Middleware (RabbitMQ): Las colas de mensajes representadas en los diagramas utilizan el broker RabbitMQ, de esta forma los componentes del sistema pueden abstraerse de las complejidades de la comunicación, como pérdidas de mensajes, congestiones, interrupciones, etc.
- Distribución de Carga por Sharding (Splits): Los componentes de redireccionamiento (*Splits*) calculan funciones de hash de manera local y determinista sobre los atributos clave (como el banco o las cuentas). Esto permite paralelizar el procesamiento entre múltiples workers independientes sin necesidad de un coordinador central que sature la red.
- Persistencia de Estado Local (Entities): Para el caso particular de la conversión de monedas a USD mediante una API (Q5), se optó por persistir los tipos de cambio ya consultados en una entidad, ya que si por cada transacción hay que realizar un llamado a la API el sistema no escala. De esta manera, cuando hay que convertir una moneda a USD primero se analiza si el tipo de cambio ya está almacenado, y en caso contrario se le solicita al API Client que realice la consulta a la API.



Arquitectura de monitoreo y elección de líder

Para garantizar la alta disponibilidad del sistema de monitoreo, se despliega un clúster de monitores redundantes coordinados mediante un Algoritmo de Elección en Anillo (Ring Election) sobre sockets TCP.

- Detección de Caída: Los monitores secundarios (followers) esperan heartbeats periódicos del líder. Si expira el HEARTBEAT_TIMEOUT, se inicia el proceso de elección (`_start_election`).
- Flujo del Algoritmo: El nodo que detecta la falla envía un mensaje `ELECTION:<MONITOR_ID>` a su sucesor interconectado.
- Tolerancia en el Anillo: Si un sucesor no responde (con timeout por variable de entorno), el algoritmo lo descarta de forma transparente e intenta conectarse con el siguiente nodo de la lista (SUCCESSORS), garantizando la continuidad del anillo ante fallos múltiples.

El monitor líder supervisa activamente la salud de todos los workers del pipeline de procesamiento.

- Protocolo de Verificación: Se realiza mediante conexiones TCP directas al puerto de salud (HEALTH_PORT, por defecto 8888). Cada worker del sistema hereda un Health server (inclusive los monitores) que maneja esto. El nodo debe responder exactamente la cadena de bytes `b"OK"`.
- Criterio de Fallo: Un componente se declara "caído" solo tras superar un límite de fallas consecutivas (MAX_FAILURES cada determinado CHECK_INTERVAL y con un timeout también por variable de entorno), evitando falsos positivos por congestión de red.

Una vez confirmada la caída anormal de un worker, el monitor líder ejecuta su recuperación automatizada interactuando directamente con el daemon de Docker.

Tolerancia a Fallas

El sistema fue diseñado para ser tolerante a fallos, a partir de las siguientes estrategias:

- Un nodo Monitor replicado que detecta las caídas de los nodos restantes, como fue explicado anteriormente.
- Persistencia de mensajes en el Middleware RabbitMQ, configurando colas y exchanges como durables. Gracias a esto, ante una caída del middleware no perdemos los mensajes.
- Reenvío de batches ante acks perdidos.
 - Rabbit espera confirmación de los nodos al enviar un mensaje. Si este no le envía un ACK por algún motivo, como una caída, Rabbit va a reintentar el envío del mensaje.
- Persistencia en común de nodos base

- Todos los nodos heredan de una clase base WorkerBase o DoubleIOWorkerBase. Estos nodos tienen un funcionamiento en común y persisten el estado básico de ejecución.
- Este estado es: EOFs recibidos, EOFs procesados, Batches procesados y el Outbox de salida.
- Ante la caída del nodo, al reiniciar recupera el estado básico propio.
- Nodos con estado propio:
 - Aggregators simples: además del estado básico, persisten los valores acumulados parciales
 - Money Converter: además del estado básico, persiste caché de cotizaciones, requests a la api pendientes, y filas ya convertidas.
 - BankNameAdder
 - BarrierFilter: además del estado básico, persiste los promedios recibidos y las transacciones recibidas antes de recibir todos los promedios.
 - Nodos de grafos para Q4: persisten edges, paths parciales y todo lo necesario para resolver la query 4.
- Estrategias de escritura atómica:
 - Para evitar que quede un estado corrupto de los archivos persistidos si es que el nodo se cae mientras escribe, se implementaron las siguientes estrategias
 - Snapshot: en nodos donde el estado persistido es chico, se hace un snapshot de el archivo de persistencia actual en un archivo temporal, y luego se hace un os.replace que es atómico.
 - Append only: en nodos donde el estado es más grande, se persiste cada registro con su largo, el payload y un checksum. Al reiniciar se recalcula el checksum y si no coincide se descarta.

Listado de tareas

Tarea	Integrante encargado
Client	Perez Esnaola Lucas
Gateway	Perez Esnaola Lucas
Protocolo interno	Todos
Protocolo externo	Todos
Filtro configurable	Perez Esnaola Lucas
Reductor de datos	Perez Esnaola Lucas
Conversor de monedas	Bustamante Gonzalo Manuel
Generador de docker compose	Bustamante Gonzalo Manuel
Agregador de subgrafos	Bustamante Gonzalo Manuel

Joiner de caminos	Bustamante Gonzalo Manuel
Contador de caminos únicos	Bustamante Gonzalo Manuel
Splitter configurable	Nemirovsky Federico
Filtro con barrera (para query 3)	Bustamante Gonzalo Manuel
Agregador configurable	Nemirovsky Federico
Validador de resultados	Nemirovsky Federico
Broker RabbitMQ	Nemirovsky Federico